STRIPE AI INFRASTRUCTURE • CERTIFICATION BLOC 2 (RNCP 41993)

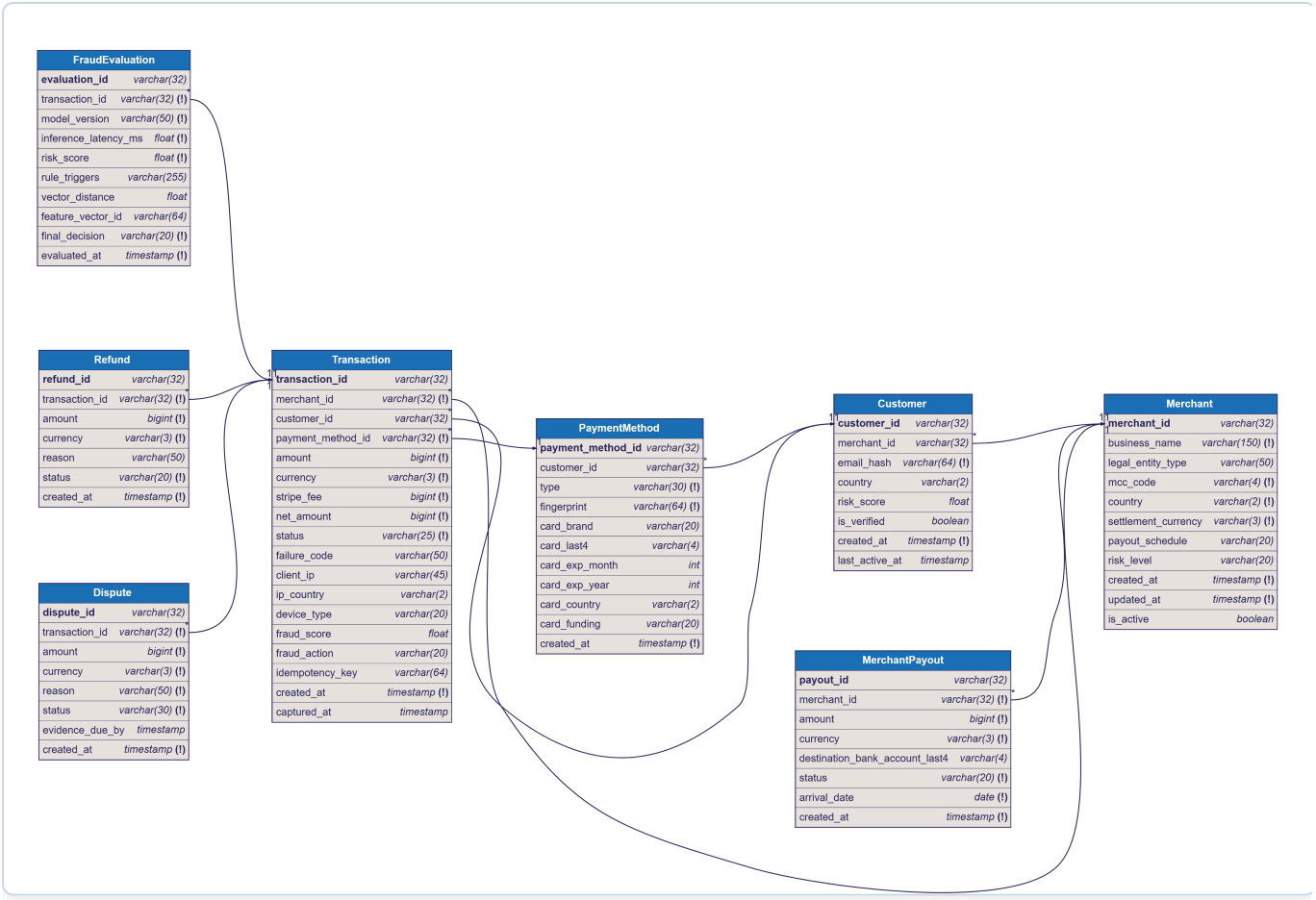
Modèle Relationnel OLTP — Transactions & Garanties ACID (3NF)

Le système transactionnel de Stripe est le cœur battant de l'activité économique de ses marchands. Il doit concilier des contraintes d'une rigueur absolue :

- **Intégrité financière sans compromis :** Chaque centime doit être imputé avec certitude (zéro perte d'argent, zéro double débit).
- **Haute cadence d'écriture :** Capacité à absorber des milliers d'écritures concurrentes par seconde lors des pics (ex : Black Friday, lancements de billetterie).
- **Normalisation en 3e Forme Normale (3NF) :** Élimination de toute anomalie de mise à jour, d'insertion ou de suppression.
- **Conformité stricte PCI-DSS v4.0 :** Zéro stockage de numéro de carte (PAN) en clair ; utilisation exclusive de tokens de paiement (`payment_method_id`).

2. Diagramme Entité-Association OLTP (ERD)

> **Modèle Relationnel Normalisé (3NF - ACID)** modélisé en **DBML** selon les standards industriels de modélisation financière (dbdiagram.io) et exporté en haute résolution.
> *Fichier source de modélisation DBML :* `stripe_oltp.dbml`
> *Script DDL SQL de création :* `stripe_oltp_olap.sql`


FraudEvaluation
evaluation_id varchar(32)
transaction_id varchar(32) (!)
model_version varchar(50) (!)
inference_latency_ms float (!)
risk_score float (!)
rule_triggers varchar(255)
vector_distance float
feature_vector_id varchar(64)
final_decision varchar(20) (!)
evaluated_at timestamp (!)
Refund
refund_id varchar(32)
transaction_id varchar(32) (!)
amount bigint (!)
currency varchar(3) (!)
reason varchar(50)
status varchar(20) (!)
created_at timestamp (!)
Dispute
dispute_id varchar(32)
transaction_id varchar(32) (!)
amount bigint (!)
currency varchar(3) (!)
reason varchar(50) (!)
status varchar(30) (!)
evidence_due_by timestamp
created_at timestamp (!)
Transaction
transaction_id varchar(32)
merchant_id varchar(32) (!)
customer_id varchar(32)
payment_method_id varchar(32) (!)
amount bigint (!)
currency varchar(3) (!)
stripe_fee bigint (!)
net_amount bigint (!)
status varchar(25) (!)
failure_code varchar(50)
client_ip varchar(45)
ip_country varchar(2)
device_type varchar(20)
fraud_score float
fraud_action varchar(20)
idempotency_key varchar(64)
created_at timestamp (!)
captured_at timestamp
PaymentMethod
payment_method_id varchar(32)
customer_id varchar(32)
type varchar(30) (!)
fingerprint varchar(64) (!)
card_brand varchar(20)
card_last4 varchar(4)
card_exp_month int
card_exp_year int
card_country varchar(2)
card_funding varchar(20)
created_at timestamp (!)
Customer
customer_id varchar(32)
merchant_id varchar(32)
email_hash varchar(64) (!)
country varchar(2)
risk_score float
is_verified boolean
created_at timestamp (!)
last_active_at timestamp
Merchant
merchant_id varchar(32)
business_name varchar(150) (!)
legal_entity_type varchar(50)
mcc_code varchar(4) (!)
country varchar(2) (!)
settlement_currency varchar(3) (!)
payout_schedule varchar(20)
risk_level varchar(20)
created_at timestamp (!)
updated_at timestamp (!)
is_active boolean
MerchantPayout
payout_id varchar(32)
merchant_id varchar(32) (!)
amount bigint (!)
currency varchar(3) (!)
destination_bank_account_last4 varchar(4)
status varchar(20) (!)
arrival_date date (!)
created_at timestamp (!)

3. Dictionnaire Détaillé des Entités Transactionnelles

3.1. `Merchant` (Marchand)

- **Rôle :** Représente l'entreprise ou l'entrepreneur titulaire du compte Stripe.
- **Points clés :**
- `mcc_code` (*Merchant Category Code*) permet d'appliquer des politiques de risque adaptées au secteur (les voyagistes ou les services de crypto-monnaies ont un risque par défaut plus élevé que les commerces de proximité).
- `settlement_currency` définit la devise de virement bancaire pour éviter les frais de change imprévus.

3.2. `Customer` (Client Final)

- **Rôle :** Acheteur final effectuant un paiement auprès d'un marchand.
- **Conformité RGPD :** L'adresse email n'est pas stockée en clair mais sous forme de hash cryptographique `email_hash` (SHA-256 avec sel) pour la détection des fraudes récurrentes sans exposer de données nominatives.

3.3. `PaymentMethod` (Moyen de Paiement Sécurisé)

- **Rôle :** Représente la carte bancaire, le compte SEPA ou le wallet (Apple Pay, Google Pay).
- **Conformité PCI-DSS :** Le numéro de carte complet (PAN) et le code de sécurité (CVV) sont interceptés à l'Edge dans un coffre-fort de tokenisation dédié (*Tokenization Vault*). La table applicative ne conserve que le token non réversible `payment_method_id`, les `card_last4` pour l'affichage au client, et l'empreinte `fingerprint` pour détecter si une même carte volée est testée sur 50 marchands différents.

3.4. `Transaction` (Table Maîtresse d'Encaissement)

- **Rôle :** Enregistrement unitaire d'un paiement, commit financier immédiat.
- **Gestion des montants :** Tous les montants (`amount`, `stripe_fee`, `net_amount`) sont rigoureusement typés en entiers (`BIGINT`) représentant des **centimes** (ex : 49,99 € \= `4999`). Cela élimine définitivement les erreurs d'arrondis et de précision inhérentes aux nombres à virgule flottante (`FLOAT` / `DOUBLE`).
- **Garantie d'Idempotence :** La colonne `idempotency_key` (index unique) permet au marchand de renvoyer la même requête réseau en cas de timeout sans risquer de débiter deux fois le client.

3.5. `FraudEvaluation` (Audit d'Inférence Radar)

- **Rôle :** Trace technique et légale de la décision d'IA prise lors de l'encaissement (\< 100 ms).
- **MLOps & Auditabilité :** Enregistre la version exacte du modèle (`model_version`), la latence constatée (`inference_latency_ms`), le score émis et les motifs de blocage. Indispensable pour la gouvernance de l'IA (AI Act) et l'explicabilité aux régulateurs bancaires.

4. Respect Rigoureux des Propriétés ACID

Propriété ACID	Problème Potentiel en Paiement	Implémentation Architecturale sur Aurora PostgreSQL
Atomicité (A)	Débit du client réussi mais échec de l'imputation du solde marchand.	Utilisation de blocs transactionnels stricts (`BEGIN ... COMMIT / ROLLBACK`). L'écriture de la transaction, le calcul des frais et la mise à jour des quotas s'exécutent en une seule opération indivisible.
Cohérence (C)	Solde négatif ou monnaie incohérente sur un remboursement.	Contraintes de vérification au niveau du moteur SQL (`CHECK (amount > 0)` , clés étrangères `REFERENCES` et contraintes d'intégrité référentielle empêchant les orphelins).
Isolation (I)	Deux requêtes concurrentes débitent simultanément le même compte (Race Condition).	Niveau d'isolation par défaut **Read Committed** avec verrous optimistes, et utilisation de **Snapshot Isolation** pour les opérations critiques, garantissant qu'aucune lecture sale (*dirty read*) ne perturbe le calcul.
Durabilité (D)	Panne de courant ou crash matériel d'un serveur juste après confirmation du paiement.	Moteur de stockage distribué d'Aurora qui réplique immédiatement le journal de transaction (*redo log*) sur **6 nœuds de stockage répartis sur 3 Zones de Disponibilité distinctes** avant d'acquitter le commit au client.

5. Stratégie d'Indexation et Performance

1. `CREATE UNIQUE INDEX idx_txns_idempotency ON transactions (idempotency_key);`
   Recherche en O(1) lors de chaque appel API pour rejeter instantanément les doublons.
2. `CREATE INDEX idx_txns_merchant_created ON transactions (merchant_id, created_at DESC);`
   Optimise les requêtes de consultation du dashboard marchand (derniers paiements).
3. `CREATE INDEX idx_pm_fingerprint ON payment_methods (fingerprint);`
   Permet au moteur de fraude de vérifier en moins de 2 ms le nombre de transactions associées à la même carte bancaire au cours des 60 dernières minutes.